

STANDAR ENDPOINT MASTER DATA

Method	Endpoint	Fungsi utama	Dipakai frontend untuk
GET	/filters/metadata	Mengambil konfigurasi filter, sort, enum, pagination, dan form metadata	Render halaman index dan form secara konsisten
GET	/summary	Mengambil ringkasan jumlah data	Card statistik di halaman index
GET	/	Mengambil list data dengan filter, search, sort, pagination	Table utama master data
POST	/	Membuat data baru	Form create
GET	/options	Mengambil data ringan untuk dropdown/select	Dropdown di form lain
GET	/id	Mengambil detail satu data	Halaman detail dan form update
PUT	/id	Mengubah data	Simpan perubahan
DELETE	/id	Soft delete data	Tombol hapus

1. GET /filters/metadata

Endpoint ini sebaiknya dipanggil pertama kali saat frontend membuka halaman master data.

Fungsinya bukan mengambil data utama, tapi mengambil konfigurasi halaman, seperti:

- default filter,
- pilihan custom period,
- opsi sorting,
- arah sorting,
- pilihan page size,
- opsi enum,
- daftar query parameter,
- metadata form create,
- metadata form update.

Di controller Service Unit, endpoint ini mengembalikan DefaultFilter, CustomPeriods, SortOptions, SortDirections, PageSizeOptions, ServiceUnitTypeOptions, QueryParameters, CreateFields, dan UpdateFields.

Contoh penggunaan frontend:

GET /api/v1/health-services/master-data/service-units/filters/metadata

Dipakai untuk:

1. *Membuat filter UI*
2. *Membuat dropdown enum*
3. *Menentukan default sort*
4. *Menentukan pilihan page size*
5. *Membaca field mana yang required*
6. *Membaca field mana yang muncul di create/update*

Untuk frontend, endpoint ini sangat penting karena membuat halaman master data lebih dinamis dan konsisten.

2. GET /summary

Endpoint ini untuk mengambil angka ringkasan.

Pada Service Unit, summary berisi:

```
TotalServiceUnit
ActiveServiceUnit
InactiveServiceUnit
RegistrationAvailableServiceUnit
KioskAvailableServiceUnit
AppointmentAvailableServiceUnit
DoctorRequiredServiceUnit
ScreeningRequiredServiceUnit
```

Controller menghitung semua data yang belum soft delete, lalu memisahkan jumlah berdasarkan status aktif dan flag operasional seperti registration, kiosk, appointment, doctor required, dan screening required.

Contoh penggunaan frontend:

GET /api/v1/health-services/master-data/service-units/summary

Dipakai untuk card seperti:

```
Total Service Unit   : 12
Active               : 10
Inactive             : 2
Available Registration : 8
Available Kiosk       : 4
Doctor Required       : 6
Screening Required    : 5
```

Untuk semua master data, summary tidak harus sama. Misalnya:

```
Master Clinic:
- TotalClinic
- ActiveClinic
- InactiveClinic
- QueueRequiredClinic
- DoctorRequiredClinic
```

```
Master Drug:
- TotalDrug
- ActiveDrug
- InactiveDrug
- LowStockPolicyDrug
- PrescriptionOnlyDrug
```

Jadi pattern endpoint-nya sama, tetapi isi summary menyesuaikan entity.

3. GET /

Endpoint ini adalah endpoint utama untuk table index.
Pada Service Unit, endpoint ini mendukung:

```
startDate
endDate
customPeriod
search
isActive
serviceUnitType
isAvailableForRegistration
isAvailableForKiosk
isAvailableForAppointment
isDoctorRequired
isScreeningRequired
sortBy
sortDirection
pageNumber
pageSize
```

Controller juga sudah melakukan paging normalisasi, filter tanggal, search ke beberapa field, filter boolean, filter enum, sorting, lalu mengembalikan PagedResult.

Contoh request:

```
GET /api/v1/health-services/master-data/service-units
?search=rawat
&isActive=true
&serviceUnitType=1
&sortBy=sortOrder
&sortDirection=asc
&pageNumber=1
&pageSize=25
```

Fungsi frontend:

1. Menampilkan table utama
2. Search data
3. Filter status aktif/nonaktif
4. Filter tipe data
5. Sort kolom
6. Pagination

Response table idealnya dibaca dari:

```
{
  "data": {
    "pageNumber": 1,
    "pageSize": 25,
    "totalData": 100,
    "totalPage": 4,
    "items": []
  }
}
```

Jadi frontend jangan hitung total page sendiri dari array items, tapi ambil dari totalPage.

4. POST /

Endpoint ini untuk create data master.

Untuk Service Unit versi terbaru, serviceUnitCode tidak perlu diisi frontend, karena dibuat otomatis oleh backend dengan pola:

```
SU-RSMMC-00001
SU-RSMMC-00002
SU-RSMMC-00003
```

Frontend cukup kirim field bisnisnya saja.

Contoh body:

```
{
  "serviceName": "Rawat Jalan",
  "serviceUnitType": 1,
  "shortName": "RAJAL",
  "locationName": "Gedung Rawat Jalan",
  "floorName": "Lantai 1",
  "isAvailableForRegistration": true,
  "isAvailableForKiosk": true,
  "isAvailableForAppointment": true,
  "isQueueRequired": true,
  "isDoctorRequired": true,
  "isScreeningRequired": true,
  "sortOrder": 1,
  "description": "Unit pelayanan rawat jalan"
}
```

Setelah create berhasil, backend mengembalikan data penting seperti:

```
Id
ServiceUnitCode
ServiceUnitName
ServiceUnitType
IsActive
```

Frontend bisa memakai response ini untuk:

1. Menampilkan toast sukses
2. Redirect ke detail
3. Redirect ke index
4. Refresh table
5. Menampilkan kode auto-generated ke user

Rekomendasi UX:

ServiceUnitCode jangan ditampilkan sebagai input wajib.
Tampilkan sebagai readonly/preview: "Auto generated by system".

5. GET /options

Endpoint ini khusus untuk dropdown, lookup, select, autocomplete, atau relasi antar master data.
Pada Service Unit, options mendukung filter:

```
serviceUnitType
isAvailableForRegistration
isAvailableForKiosk
isAvailableForAppointment
onlyActive
search
```

Secara default onlyActive = true, jadi frontend akan mendapat data aktif saja.

Contoh request untuk dropdown registrasi:

```
GET /api/v1/health-services/master-data/service-units/options
?onlyActive=true
&isAvailableForRegistration=true
```

Contoh request untuk dropdown kiosk:

```
GET /api/v1/health-services/master-data/service-units/options
?onlyActive=true
&isAvailableForKiosk=true
```

Endpoint ini berbeda dari GET /.

GET / untuk table lengkap dan pagination.

GET /options untuk dropdown ringan.

Jadi frontend jangan pakai endpoint list utama untuk dropdown, karena response-nya lebih berat.

6. GET /{id}

Endpoint ini untuk mengambil detail satu data.

Pada Service Unit, detail mengembalikan data yang lebih lengkap dibanding list, termasuk Description.

Contoh request:

GET /api/v1/health-services/master-data/service-units/{id}

Dipakai untuk:

1. Halaman detail
2. Mengisi form update
3. View mode/read-only
4. Validasi data sebelum edit

Kalau data tidak ditemukan atau sudah soft delete, backend mengembalikan 404.

Frontend sebaiknya handle:

404 → tampilkan "Data tidak ditemukan atau sudah dihapus."

7. PUT /{id}

Endpoint ini untuk update data.

Pada Service Unit, update mencari data berdasarkan Id dan memastikan data belum soft delete. Setelah itu backend update field bisnis seperti nama, tipe, lokasi, flag rule, sort order, description, dan status aktif.

Contoh request:

PUT /api/v1/health-services/master-data/service-units/{id}

Contoh body:

```
{
  "serviceUnitName": "Rawat Jalan Eksekutif",
  "serviceUnitType": 1,
  "shortName": "RAJAL-EKS",
  "locationName": "Gedung A",
  "floorName": "Lantai 2",
  "isAvailableForRegistration": true,
  "isAvailableForKiosk": false,
  "isAvailableForAppointment": true,
  "isQueueRequired": true,
  "isDoctorRequired": true,
  "isScreeningRequired": true,
  "sortOrder": 2,
  "description": "Unit pelayanan rawat jalan eksekutif",
  "isActive": true
}
```

Untuk frontend:

1. Ambil data dulu lewat GET `/id`
2. Isi form update
3. User ubah data
4. Submit ke PUT `/id`
5. Jika sukses, redirect ke index atau detail

Untuk master data, PUT sebaiknya dianggap sebagai full update, bukan partial update. Kalau nanti butuh update status saja, bisa dibuat endpoint tambahan seperti:

```
PATCH /id/activate  
PATCH /id/deactivate
```

Tapi untuk standar awal, PUT `/id` sudah cukup.

8. DELETE `/id`

Endpoint ini untuk soft delete, bukan hard delete.
Pada Service Unit, delete akan:

```
IsDelete = true  
IsActive = false  
DeleteDateTime = DateTime.UtcNow  
DeleteBy = current user
```

Controller juga melakukan validasi: jika Service Unit sudah digunakan oleh Clinic, maka tidak boleh dihapus.

Contoh request:

DELETE `/api/v1/health-services/master-data/service-units/id`

Frontend harus handle beberapa kondisi:

```
200 → data berhasil dihapus  
400 → data tidak bisa dihapus karena sudah digunakan  
404 → data tidak ditemukan  
401 → belum login  
403 → tidak punya akses
```

Untuk master data rumah sakit, rule seperti ini penting. Data master yang sudah dipakai transaksi atau relasi master lain jangan sembarang dihapus.